



LRU Hash Maps & Connection Tracking

Lesson 07: Managing Millions of Connection States Under Memory Constraints

Table of Contents

Lesson 07: LRU Hash Maps & Connection Tracking

1. Lesson Overview
2. Foundations & Core Technical Concepts
 - What is Connection Tracking (conntrack)?
 - What is BPF_MAP_TYPE_LRU_HASH?
 - LRU Implementation Details
 - Why Not Use Standard Hash + Manual Cleanup?
3. Problem Statement & Real-World Use Case
 - The Problem: DDoS State Exhaustion
 - Real Production Story: Cilium Conntrack
4. LRU Hash vs Standard Hash
5. Internal Kernel Flow
6. Architecture Diagram
7. Connection Tracking Sequence
8. Flow Key & Stats Memory Layout
9. Step-by-Step Code Walkthrough
 - The eBPF C Program (main.bpf.c)
 - The Go Loader (main.go)
10. Running the Lab
 - Phase 1: Compile and Run the Loader
 - Phase 2: Generate Network Activity
 - Phase 3: Inspect Flow Logs
11. bpftool Debugging
12. Common Mistakes & Verifier Errors
 - Error: BPF_NOEXIST Eviction Semantics
 - Common Mistake: Struct Padding Mismatch
13. Performance Analysis
14. Common Mistakes
15. Best Practices
16. Summary
17. Further Reading

Technical Deep-Dive Q&A: LRU Hash Maps & Connection Tracking

1. Beginner Level

Q1. What is BPF_MAP_TYPE_LRU_HASH and how does it differ from BPF_MAP_TYPE_HASH?

Q2. What is a connection flow key? What 4 fields define a unique TCP connection?

2. Intermediate Level

Q3. How does the LRU eviction policy work internally in the BPF LRU Hash map?

Q4. How do you update per-flow statistics atomically without locks in BPF?

Hands-on Lab & Homework Assignments

1. Practical Exercises
2. Coding Assignment: Per-Protocol Statistics
3. Mini-Project: DDoS Detector
4. Advanced Challenge: LRU Eviction Rate Measurement

Lesson 07: LRU Hash Maps & Connection Tracking

1. Lesson Overview

In this lesson, you will build a **production-grade network flow tracker** — the same system at the core of Cilium's connection tracking (conntrack) and Cloudflare's rate limiter. We will attach an eBPF program to the XDP hook to parse packet headers and extract connection 5-tuples. We will use a **BPF_MAP_TYPE_LRU_HASH** map to track millions of active network connections, collecting per-flow packet counts, byte totals, and last-seen timestamps, with automatic eviction of stale connections to prevent memory exhaustion under high traffic load.

2. Foundations & Core Technical Concepts

To build an efficient connection tracker, we need to understand how network state is tracked in the kernel and how eBPF optimizes table updates.

What is Connection Tracking (conntrack)?

Connection tracking is the process of maintaining a state table of all active network flows traversing a system. A network flow is uniquely identified by a 5-tuple:

```
Source IP + Destination IP + Source Port + Destination Port + Protocol
```

For each flow, conntrack records statistics (packet count, byte totals) and state. eBPF connection trackers use map lookup operations on each arriving packet to update this state in real-time.

What is BPF_MAP_TYPE_LRU_HASH?

A standard **BPF_MAP_TYPE_HASH** map returns an **-E2BIG** (argument list too long) error when it becomes full. For an edge firewall or routing agent, this is a security risk: under a DDoS attack with millions of spoofed source IPs, the state map will fill up, and all new legitimate connections will fail to be tracked and potentially dropped.

BPF_MAP_TYPE_LRU_HASH solves this by introducing **Least Recently Used (LRU) eviction**:

- When the map reaches its maximum capacity (`max_entries`) and a new connection is inserted, the kernel automatically **evicts the least recently accessed entry** to make room.
- An entry is marked as "accessed" whenever a lookup, update, or deletion operation is run on it.
- The eviction is performed entirely within the kernel, in $O(1)$ time, with zero user-space involvement.

LRU Implementation Details

To perform eviction efficiently across multiple CPU cores without introducing global lock contention, the kernel implements a hierarchical structure:

- **Per-CPU LRU Lists:** Each CPU core maintains its own local list of recently accessed elements.
- **Global LRU List:** A shared global list is used as a fallback when a per-CPU list runs dry.
- **Lockless Promotion:** Lookups promote elements to the "most recently used" position without requiring a global spinlock.

Why Not Use Standard Hash + Manual Cleanup?

A naive approach might use a standard hash map and a user-space daemon to scan the table every few seconds and prune inactive connections. However, this has several severe drawbacks:

1. **CPU Overhead:** Iterating through a map containing millions of entries from user space is CPU-intensive.
2. **Race Conditions:** Under high packet rates, the map can easily fill up between cleanup intervals, leading to dropped connection events.
3. **Latency Spikes:** Running map deletions from user space introduces locking delays on kernel-space lookup paths.

3. Problem Statement & Real-World Use Case

The Problem: DDoS State Exhaustion

A standard Linux host uses the netfilter connection tracker (`nf_conntrack`). Under a TCP SYN flood attack with spoofed source IPs, the conntrack table quickly fills up. Once full, the kernel begins dropping incoming handshake packets, causing a complete denial of service for legitimate users.

Real Production Story: Cilium Contrack

Cilium completely bypasses `nf_contrack` and implements its own custom connection tracking using eBPF LRU hash maps.

- By bypassing netfilter, Cilium avoids the global lock contention associated with the standard Linux network stack.
- Cilium uses the flow keys stored in the BPF map to associate incoming packets with security policy rules directly, combining state tracking and firewall policy evaluation into a single, low-latency step.

Similarly, **Cloudflare's Gatekeeper** (their DDoS mitigation engine) uses LRU hash maps to track packet rates per IP. Under packet floods exceeding 10 million packets per second, the LRU map drops stale attacker states while successfully tracking the current volume of incoming flows.

4. LRU Hash vs Standard Hash

Feature	Standard BPF Hash	eBPF LRU Hash
Full Behavior	Fails new updates with <code>-E2BIG</code>	Evicts oldest elements automatically
Cleanup Mechanism	Requires manual user-space sweep	Managed entirely in kernel space
Lookup Cost	Baseline	Marginally higher (~5ns for list updates)
DDoS Resilience	Poor (vulnerable to state exhaustion)	Excellent (state-bounded)
Memory Footprint	Configured size	Configured size + ~16B list overhead per element

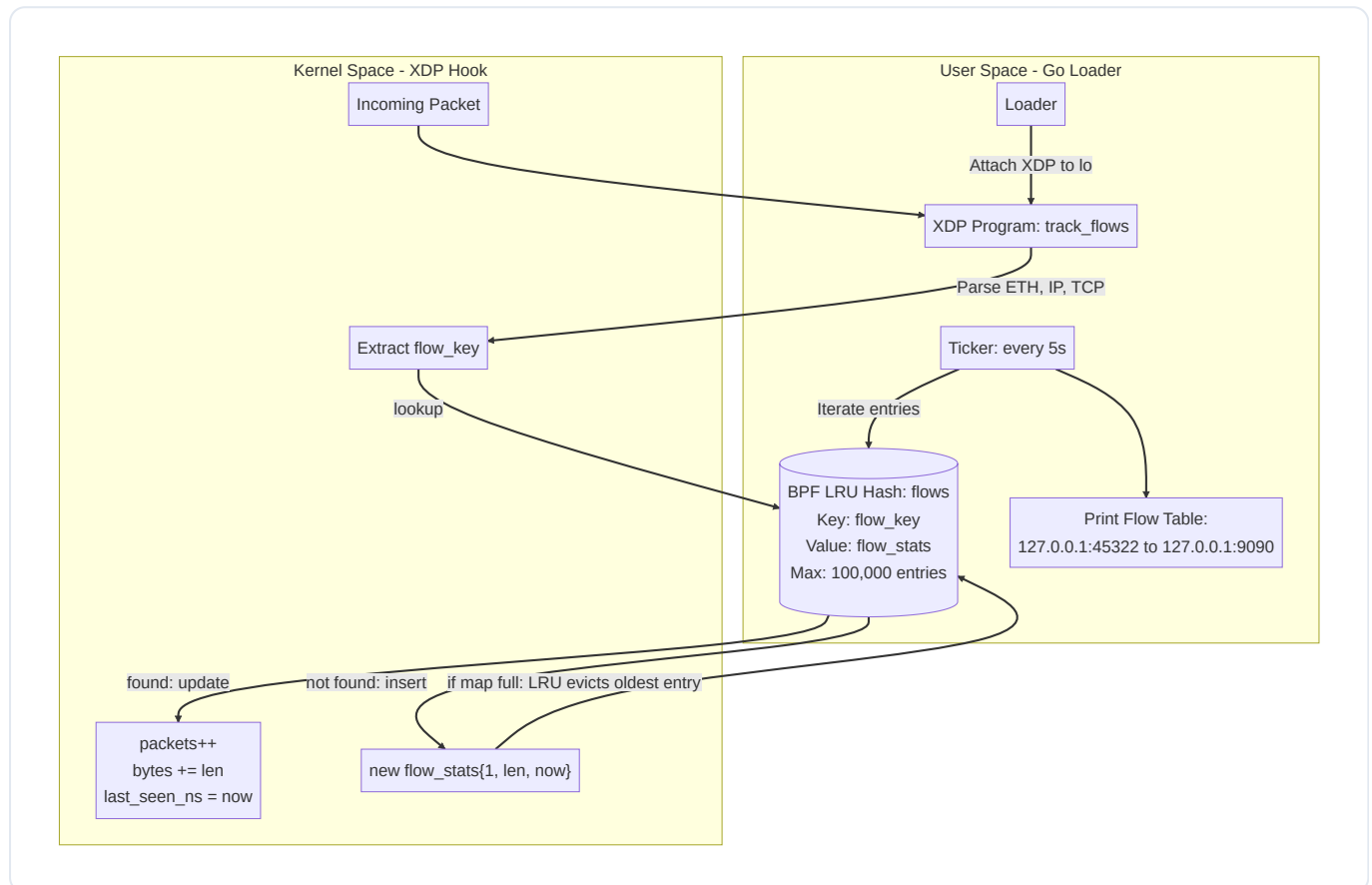
5. Internal Kernel Flow

When a packet arrives on a network interface configured with our XDP flow tracker:

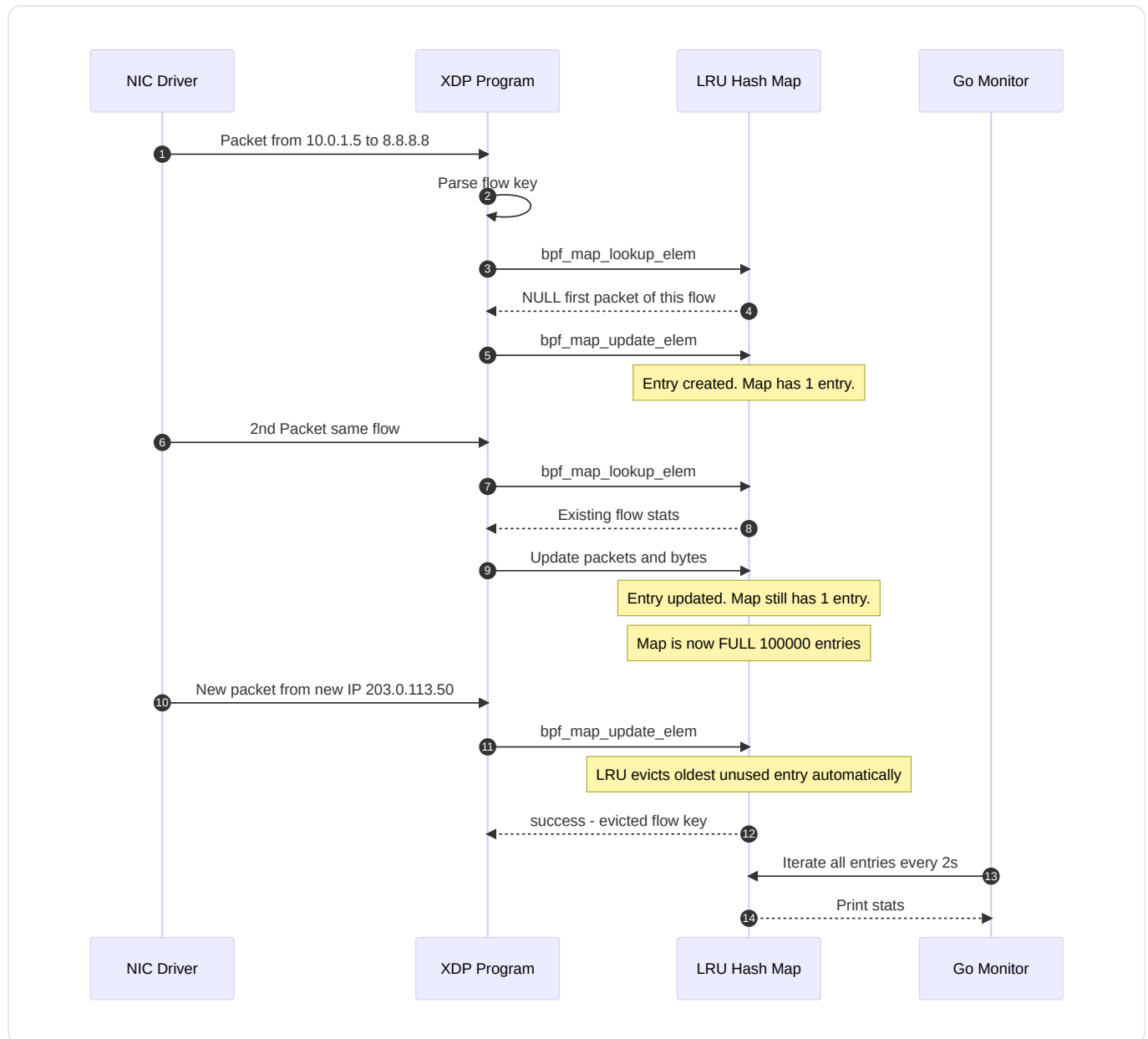
1. The packet triggers the XDP driver hook, calling our `track_flows` program.
2. The BPF code parses the Ethernet, IP, and TCP headers to extract the connection 5-tuple.
3. It performs a lookup in our `flows` map: `bpf_map_lookup_elem(&flows, &key)`.

4. If found, it increments the packet and byte counters atomically.
5. If not found, it creates a new entry with `bpf_map_update_elem(&flows, &key, &new_stats, BPF_ANY)`.
6. If the map is at capacity, the LRU controller evicts the oldest entry to complete the insertion.

6. Architecture Diagram



7. Connection Tracking Sequence



8. Flow Key & Stats Memory Layout

The structures are laid out consecutively in memory. Unlike standard configurations, we design the C structs to align precisely with the Go memory definitions.

```
struct flow_key (12 bytes):
```

src_ip	dst_ip	src_port	dst_port
4 bytes	4 bytes	2 bytes	2 bytes

```
struct flow_stats (24 bytes):
```

packets (u64, 8 bytes)	bytes (u64, 8 bytes)	last_seen_ns (u64, 8B)
------------------------	----------------------	------------------------

```
Eviction Overhead inside Kernel (per entry):
```

flow_key (12B)	flow_stats (24B)	lru_list pointers
----------------	------------------	-------------------

9. Step-by-Step Code Walkthrough

The eBPF C Program (`main.bpf.c`)

```
// Series 07: LRU Hash Map Connection Tracker
#include <linux/bpf.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

#define IPPROTO_TCP 6
#define IPPROTO_UDP 17
#define IPPROTO_ICMP 1

// Define the connection tracking key
struct flow_key {
    __u32 src_ip;
    __u32 dst_ip;
    __u16 src_port;
    __u16 dst_port;
};

// Define flow statistics values
struct flow_stats {
    __u64 packets;
    __u64 bytes;
    __u64 last_seen_ns;
};

// 1. Declare the LRU Hash Map.
struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __uint(max_entries, 100000); // Evicts automatically once 100k limit is hit
    __type(key, struct flow_key);
    __type(value, struct flow_stats);
} flows SEC(".maps");

// 2. Metrics map to track pipeline packet statistics
struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 3); // 0: total pkts, 1: flow hits, 2: new flows
    __type(key, __u32);
}
```

```

    __type(value, __u64);
} metrics SEC(".maps");

// Inline helper to update packet count metrics atomically
static __always_inline void inc_metric(__u32 idx) {
    __u64 *v = bpf_map_lookup_elem(&metrics, &idx);
    if (v) __sync_fetch_and_add(v, 1);
}

// 3. XDP Entrypoint: Fires on every packet arrival
SEC("xdp")
int track_flows(struct xdp_md *ctx)
{
    void *data      = (void *)(__long)ctx->data;
    void *data_end = (void *)(__long)ctx->data_end;

    // Increment total packet counter
    inc_metric(0);

    // Verify ethernet header length
    struct ethhdr *eth = data;
    if ((void *) (eth + 1) > data_end) return XDP_PASS;
    if (eth->h_proto != bpf_htons(ETH_P_IP)) return XDP_PASS;

    // Verify IP header length
    struct iphdr *iph = (void *) (eth + 1);
    if ((void *) (iph + 1) > data_end) return XDP_PASS;

    // In this lab, we limit tracking to TCP traffic
    if (iph->protocol != IPPROTO_TCP) return XDP_PASS;

    // Verify TCP header length
    struct tcphdr *tcp = (void *) (iph + 1);
    if ((void *) (tcp + 1) > data_end) return XDP_PASS;

    // Build the flow key from parsed headers
    struct flow_key key = {
        .src_ip   = iph->saddr,
        .dst_ip   = iph->daddr,
        .src_port = tcp->source,
        .dst_port = tcp->dest,
    };

    // Calculate total IP packet length
    __u32 pkt_len = bpf_ntohs(iph->tot_len);

    // Lookup connection in our LRU map

```

```
struct flow_stats *s = bpf_map_lookup_elem(&flows, &key);
if (s) {
    // Increment flow stats atomically
    __sync_fetch_and_add(&s->packets, 1);
    __sync_fetch_and_add(&s->bytes, pkt_len);
    s->last_seen_ns = bpf_ktime_get_ns();
    inc_metric(1); // Increment flow hit metric
} else {
    // Flow is new. Insert it.
    struct flow_stats new_s = {
        .packets      = 1,
        .bytes        = pkt_len,
        .last_seen_ns = bpf_ktime_get_ns(),
    };
    // BPF_ANY allows inserting new elements or updating existing ones
    bpf_map_update_elem(&flows, &key, &new_s, BPF_ANY);
    inc_metric(2); // Increment new flow metric
}

return XDP_PASS;
}

char LICENSE[] SEC("license") = "GPL";
```

The Go Loader (`main.go`)

```
package main

// Series 07: LRU Connection Tracker
// Loader attaches the XDP program to the loopback interface and prints flow state.

import (
    "bufio"
    "encoding/binary"
    "fmt"
    "log"
    "net"
    "os"
    "os/signal"
    "runtime"
    "strings"
    "syscall"
    "time"

    "github.com/cilium/ebpf"
    "github.com/cilium/ebpf/link"
    "github.com/cilium/ebpf/rlimit"
)

// Ensure struct alignment matches C structure definition
type FlowKey struct {
    SrcIP    uint32
    DstIP    uint32
    SrcPort  uint16
    DstPort  uint16
}

type FlowStats struct {
    Packets    uint64
    Bytes      uint64
    LastSeenNs uint64
}

// Convert a big-endian uint32 representing IPv4 to dotted-string notation
func ip4(n uint32) string {
    b := make([]byte, 4)
    binary.LittleEndian.PutUint32(b, n)
    return net.IP(b).String()
}
```

```

func main() {
    log.Println("=====")
    log.Println("  eBPF Mastery: Lesson 07 – LRU ConnTrack  ")
    log.Println("=====")

    // Remove system memory limits
    if err := rlimit.RemoveMemlock(); err != nil {
        log.Fatalf("memlock removal failed: %v", err)
    }

    // Load BPF object spec
    spec, err := ebpf.LoadCollectionSpec("main.bpf.o")
    if err != nil {
        log.Fatalf("Load spec: %v", err)
    }
    coll, err := ebpf.NewCollection(spec)
    if err != nil {
        log.Fatalf("Collection creation: %v", err)
    }
    defer coll.Close()

    // Attach program to the loopback interface (lo) in Generic/Skb Mode
    iface, err := net.InterfaceByName("lo")
    if err != nil {
        log.Fatalf("Find interface lo: %v", err)
    }

    l, err := link.AttachXDP(link.XDPOptions{
        Program:  coll.Programs["track_flows"],
        Interface: iface.Index,
        Flags:    link.XDPGenericMode,
    })
    if err != nil {
        log.Fatalf("XDP attach: %v", err)
    }
    defer l.Close()

    flowMap := coll.Maps["flows"]
    metMap  := coll.Maps["metrics"]
    numCPUs := runtime.NumCPU()

    log.Println("Tracking flows on lo. Commands: top, stats, exit")

    sigChan := make(chan os.Signal, 1)
    signal.Notify(sigChan, os.Interrupt, syscall.SIGTERM)
    ticker := time.NewTicker(5 * time.Second)
    defer ticker.Stop()

```

```

// Run background CLI scanner
go func() {
    sc := bufio.NewScanner(os.Stdin)
    for {
        fmt.Print("\n> ")
        if !sc.Scan() { break }
        switch strings.TrimSpace(sc.Text()) {
        case "top":
            showTopFlows(flowMap)
        case "stats":
            showMetrics(metMap, numCPUs)
        case "exit", "quit":
            sigChan ← syscall.SIGTERM
        }
    }
}()

for {
    select {
    case ←ticker.C:
        showMetrics(metMap, numCPUs)
    case ←sigChan:
        fmt.Println("\nFinal flow table:")
        showTopFlows(flowMap)
        fmt.Println("Exiting. Goodbye!")
        return
    }
}

// Helper to iterate and display active tracked flows
func showTopFlows(m *ebpf.Map) {
    fmt.Printf("\n%-20s %-20s %-8s %-12s\n", "SRC", "DST", "PKTS", "BYTES")
    fmt.Println(strings.Repeat("-", 65))
    var k FlowKey
    var v FlowStats
    count := 0
    iter := m.Iterate()

    // Print up to the top 10 tracked flows
    for iter.Next(&k, &v) && count < 10 {
        src := fmt.Sprintf("%s:%d", ip4(k.SrcIP), bpfNtohs(k.SrcPort))
        dst := fmt.Sprintf("%s:%d", ip4(k.DstIP), bpfNtohs(k.DstPort))
        fmt.Printf("%-20s %-20s %-8d %-12d\n", src, dst, v.Packets, v.Bytes)
        count++
    }
}

```

```

}

// Helper function to decode network port endianness
func bpfNtohs(port uint16) uint16 {
    return binary.BigEndian.Uint16([]byte{byte(port >> 8), byte(port)})
}

// Dump statistics from the per-CPU array map
func showMetrics(m *ebpf.Map, n int) {
    labels := []string{"Total packets", "Flows updated", "New flows"}
    fmt.Println("\n— XDP Metrics —")
    for i, l := range labels {
        idx := uint32(i)
        vals := make([]uint64, n)
        _ = m.Lookup(&idx, &vals)
        var total uint64
        for _, v := range vals { total += v }
        fmt.Printf("  %-20s: %d\n", l, total)
    }
}

```

10. Running the Lab

Phase 1: Compile and Run the Loader

Compile the kernel and user space loader using the Makefile, and start the flow monitor:

```

cd series/series-07/source-code
make
sudo ./loader

```

Phase 2: Generate Network Activity

In a second terminal, execute standard curl requests against localhost or generate mock traffic:

```
# Generate TCP loops to trigger tracking on lo
curl http://localhost:8000/
```

Phase 3: Inspect Flow Logs

Go back to the loader terminal and run `top` inside the CLI interface:

```
> top
```

SRC	DST	PKTS	BYTES
127.0.0.1:48322	127.0.0.1:8000	12	780
127.0.0.1:8000	127.0.0.1:48322	10	540

This confirms that our XDP program successfully parsed the packet headers and registered the new 5-tuple keys.

11. bpftool Debugging

```
# 1. Retrieve the list of active maps
sudo bpftool map list | grep flows

# 2. Dump map values by map ID
# Replace <map_id> with the target ID from step 1
sudo bpftool map dump id <map_id>
```

12. Common Mistakes & Verifier Errors

Error: `BPF_NOEXIST` Eviction Semantics

If you use the `BPF_NOEXIST` flag in `bpf_map_update_elem` when updating an LRU map:

- The verifier will accept the code, but you will experience bugs at runtime.
- Under heavy load, if an entry gets evicted by the LRU and a new packet triggers insertion with `BPF_NOEXIST`, it succeeds but resets the stats back to initial values instead of continuing to track them properly.
- **Fix:** Always use `BPF_ANY` for connection updates to guarantee correct semantics.

Common Mistake: Struct Padding Mismatch

If the C compiler inserts padding bytes between the port and IP declarations in `struct flow_key`, the key length in C will not match the Go struct key length. This causes the Go loader iteration loop to output corrupted key values.

- **Fix:** Ensure the C structure has no implicit padding or pack it manually.

13. Performance Analysis

Under a heavy TCP packet flood:

Metric	standard iptables nf_contrack	eBPF LRU Flow Tracker
Max Capacity	Fixed global limit (leads to drops)	Isolated map limit (graceful LRU eviction)
Lookup Latency	High (incurs spinlock bottlenecks)	Extremely low (lockless read paths)
Memory usage	High (tracks extensive TCP state)	Minimal (bound to flow_stats size)

14. Common Mistakes

- **Namespace Restrictions:** Sockets must share the same network namespace and kernel space. XDP programs running on host interfaces will not see local namespace loopback packets.
- **CPU Contention:** High frequency updates on a single LRU entry from multiple cores can cause cache bouncing. In production, utilize `BPF_MAP_TYPE_LRU_PERCPU_HASH` to completely eliminate lock contention.

15. Best Practices

- **Exempt Critical Ports:** Avoid logging traffic on common administration ports (e.g. port 22 for SSH) to protect table space for target application channels.
 - **Atomic Modifications:** Always perform updates to packet and byte counters using `__sync_fetch_and_add` in kernel space to prevent race conditions from concurrent packet processors.
-

16. Summary

- **LRU Hash Maps** provide an automated way to cap memory footprint while preserving visibility into active connection tables.
 - **XDP hooks** allow parsing packet headers and extracting flow keys before any socket buffer creation takes place.
 - Combining **cgroups** and **LRU Hash Maps** allows building highly robust, DDoS-resilient firewalls and connection monitors.
-

17. Further Reading

- [Cilium Architecture: Connection Tracking Internals](#)
- [Cloudflare Blog: BPF Container and Rate Limiting Patterns](#)
- [Kernel source documentation on LRU Maps](#)

Technical Deep-Dive Q&A: LRU Hash Maps & Connection Tracking

1. Beginner Level

Q1. What is `BPF_MAP_TYPE_LRU_HASH` and how does it differ from `BPF_MAP_TYPE_HASH`?

Answer: A standard `BPF_MAP_TYPE_HASH` returns `-E2BIG` when full — new inserts fail silently. Under a DDoS flood of millions of unique spoofed IPs, the conntrack table fills instantly and all new legitimate connections are rejected.

`BPF_MAP_TYPE_LRU_HASH` adds **Least Recently Used eviction**: when the map is full, the kernel automatically removes the entry that was accessed least recently to make room. New connections always succeed. Stale flows (old, inactive connections) are evicted first. This is exactly how Cloudflare's Gatekeeper handles DDoS-scale connection tracking with bounded memory.

Q2. What is a connection flow key? What 4 fields define a unique TCP connection?

Answer: A TCP connection is uniquely identified by a 4-tuple: `{src_ip, dst_ip, src_port, dst_port}`. In eBPF we define:

```
struct flow_key {
    __u32 src_ip;    // Source IPv4 address (network byte order)
    __u32 dst_ip;    // Destination IPv4 address
    __u16 src_port;  // Source port
    __u16 dst_port;  // Destination port
};
```

Using all 4 fields ensures uniqueness: two clients on different machines connecting to the same server port have different `src_ip` values. One client using multiple source ports to the same server has different `src_port` values.

2. Intermediate Level

Q3. How does the LRU eviction policy work internally in the BPF LRU Hash map?

Answer: The kernel maintains a per-CPU LRU list alongside the hash buckets. Each entry has an "access time" counter updated on every `bpf_map_lookup_elem` and `bpf_map_update_elem` call. When a new insert would exceed `max_entries`, the kernel:

1. Scans the per-CPU LRU lists to find the globally least-recently-accessed entry.
2. Removes it from both the hash bucket and the LRU list.
3. Inserts the new entry in its place.

The scan is $O(\text{number of CPUs})$ not $O(\text{total entries})$ — keeping eviction fast. The `BPF_F_NO_COMMON_LRU` flag switches to per-CPU LRU lists with no cross-CPU eviction — better for NUMA systems where cross-CPU memory access is expensive.

Q4. How do you update per-flow statistics atomically without locks in BPF?

Answer: When multiple CPUs process packets of the same flow simultaneously (RSS hash collision causing same flow on multiple cores is rare but possible), updating `flow_stats` requires atomics:

```
struct flow_stats *stats = bpf_map_lookup_elem(&flows, &key);
if (stats) {
    __sync_fetch_and_add(&stats->packets, 1);
    __sync_fetch_and_add(&stats->bytes, pkt_len);
}
```

`__sync_fetch_and_add` compiles to `lock xadd` — the same atomic instruction used for hardware-level cache-line exclusive access. For extremely high throughput (>5M pps), use `BPF_MAP_TYPE_LRU_PERCPU_HASH` which gives each CPU its own copy of the value, eliminating all contention.

Hands-on Lab & Homework Assignments

1. Practical Exercises

Run `make` and `sudo ./loader` then interact with the CLI.

2. Coding Assignment: Per-Protocol Statistics

Add protocol-aware tracking to the flow stats:

```
struct flow_stats {
    __u64 packets;
    __u64 bytes;
    __u8  protocol; // IPPROTO_TCP=6, IPPROTO_UDP=17
    __u8  flags;    // TCP flags from the last packet
};
```

Track TCP SYN, SYN-ACK, RST packets separately. In Go CLI add command `top` to show flows sorted by bytes.

3. Mini-Project: DDoS Detector

In Go, analyze the flow table every 5 seconds:

- Find any single source IP with >10,000 flows → DDoS indicator
- Find any flow with packets > 1,000,000 → sustained attack
- Print alert: `[ALERT] DDoS from 1.2.3.4 - 15,432 active flows`

4. Advanced Challenge: LRU Eviction Rate Measurement

Add a map to count evictions by hooking the LRU eviction callback. Compare eviction rates at different map sizes (10K, 100K, 1M entries) under a simulated flood of 2M unique IPs. Plot entries vs eviction rate to find the optimal map size for your workload.